

DCIT 208 IA 1

STUDENT NAME: JASON NHYIRA AGYEMAN

STUDENT ID 22323887

TEAM NAME: CODE GETTERS

PROJECT TITLE: CHURCH WEBAPP

TEAM REPOSITORY: <https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-client-project-2026-team-engineering-repository-seg26-50-codegetter.git>

INDIVIDUAL REPOSITORY:
https://github.com/itsjasonagyeman/DCIT_208_personal_repo.git

YOUTUBE LINK: <https://youtu.be/JJwl22cVnoM>

DATE: 26TH JUNE, 2026

AI DISCLOSURE: I used / did not use AI assistance. If used, the tool, purpose, prompt summary, output accepted/rejected and verification steps are documented. I understand that I must be able to explain and defend every part of this submission

22323887

Jason Nhyira Agyeman

Code Getters

26th June, 2026

11:21 AM

EVENT PROCESSING

Admin Flow

- Short one sentence flow statement:

- Admin should have the ability to create an event,

store with in a database and get a successful JSON response

Flow Diagram

Admin Flow Diagram

Admin



Browser / Mobile

Admin Dashboard



Admin Domain

(admin.internationalcentreforprayer.org)



DNS resolution and

Internet routing (successful)



HTTPS POST request



API Server Interaction (FAST API)



Application Logic (request validation, admin authentication, event object creation)



Database Event Storage



JSON Response ('Event Successfully created')



Admin sees success

User Flow Diagram

Visitor



Browser / mobile



Church Domain

(internationalcentreforprayer.org/events)



DNS resolution and Internet routing (successful)



HTTP GET / POST request



Application Logic, Event fetching, payment processing
(paystack / stripe)



Ticket purchase saved in database.



JSON response ('Ticket successfully purchased')



User sees ticket saved.

POSSIBLE FAILURE POINT

- Multiple users may want to purchase a limited supply of tickets at the same time.

POSSIBLE SECURITY CONCERN

- Poor admin / user authentication.



Scanned with CamScanner

SYSTEM DESIGN QUESTIONNAIRE

1. The user (in this instance both the admin and the visitor) may be on any device that has the ability to display the webapp, including but not limited to mobile phones and laptop screens. Ideally the web app will be responsive enough to cater for most common screen devices. This is

important for user experience in the sense that the web app is easy to use and navigate on multiple devices.

2. It is a browser application. The browser application is specifically chosen in order to be able to quickly push to market without having to go through the strenuous process involved in mobile application requirement certification by apple and google. This also helps with quick testing and reduces overhead in having to cater for different Operating Systems, which would drastically increase development time. With a browser-based application we can simply create one codebase and be sure it works on multiple devices.
3. The URL most likely to be used is 'internationalcentreforprayer.org' as that is the name of the client's organization. On the part of the admin, the URL likely to be used is 'admin.internationalcentreforprayer.org'. Visitors access information through specific endpoints such as 'internationalcentreforprayer.org/events'. These URLs on the backend are mapped to API endpoints, through which data can be managed and secured easily.
4. The protocol that is safest to use for this particular project will be an HTTPS protocol as it is industry standard and widely used. It is widely used because it ensures that all data transmitted between the server and user is encrypted.
5. Data sent to the server from the admin standpoint is one that includes the name of the event, preview pictures/flyers (which are stored separately and are referenced through IDs), description of the event, available tickets/ seats and pricing. On the user end the data sent is the user's information (i.e. The user id, for reference etc.).
6. Multiple metrics validate the request. With fastAPI in conjunction with pydantic JSON validation is made easier, such that the information we request and receive matches the schema provided, else an error is thrown. For user validation, firebase api (which the team plans on using right now) is most plausible. This is meant to ensure data protection and data integrity.
7. The Events table will be altered since there is a new POST request for it. The User will be linked to a particular purchased event through foreign keys so the user table itself won't be changed, since it statically stores user information.

8. An expected successful JSON response is expected from the CRUD requests. So, for instance a response such as 'Event successfully created is expected'. These responses help the frontend display messages to the user that confirm ticket/ event purchased .
9. If validation fails, the API returns an HTTP 400 (Bad Request) or HTTP 422 (Unprocessable Entity) response containing details of the validation error. The front end then displays an appropriate error message to the user. This error handling is extremely important because it ensures that poor data isn't sent to the database.
10. If the database is down an error would be thrown (500 internal server error) at that instance, the frontend when it receives this error through application logic will display a separate screen relaying a server down message to general users.
11. If 500 users try to buy it at once, here's what might happen, the system registers tickets for this event to all 500 users. That is a problem. A possible solution is to update the ticket property only after all validation checks are complete.
12. One major privacy concern is that users may be able to gain access to another user's ticket history in the instance of poor firebase integration. This has the potential to lead to very serious data leaks.

AI USE

DATE	TOOL	PROMPT	OUTPUT USED	OUTPUT REJECTED	VERIFICATION METHOD
6/26/2026	ChatGPT	DNS resolution on a request-flow diagram	How it works in the diagram (Assumed as successful)	Information on how DNS work and	Google search (https://www.cloudflare.com/learning/dns/what-is-dns)
6/26/2026	ChatGPT	Review System design	Minor corrections to technical terms (like changes from database to server)	Everything else	Human Verified (i.e. Me)
6/26/2026	ChatGPT	Increase robustness of wording to reach 900 words	Nothing copied and pasted a lot of paraphrasing	Almost everything	Human Verified

